Bölüm 2: Karar ve Döngü Yapıları

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: week3_partI_Control
7 Kasım 2026

İçindekiler

Bölüm 2 · Karar ve Döngü Yapıları 3

Bölümün amacı 3

Konu anlatımı 3

Hatırlatma: if-else ile fazla mesai (Slayt 1–3) 3

İç içe karar yapıları (Slayt 4–9) 3

if-elif-else (Slayt 10–15) 4

Tekrar yapıları ve while (Slayt 16–21) 4

for döngüsü ve range (Slayt 22–35) 5

Birikimli toplam ve artırılmış atama (Slayt 36–38) 6

Sentinel ve girdi doğrulama (Slayt 39–41) 6

break, continue ve pass (Slayt 42–50) 7

İç içe döngüler (Slayt 51–56) 7

Sık yapılan hatalar 8

Kısa tekrar soruları 8

Cevaplar 9

Bölüm 2 · Karar ve Döngü Yapıları

Bölümün amacı

Bu bölüm programın akışını kontrol etmeyi öğretir: **birden çok koşula göre karar vermek** ve **bir işi tekrarlatmak**. Sonunda şunları yapabilmelisin:

- İç içe karar yapıları ve `if-elif-else` yazmak
- Koşul kontrollü (`while`) ve sayı kontrollü (`for` + `range`) döngüler kurmak
- Birikimli toplam, artırılmış atama, sentinel ve girdi doğrulama kalıplarını kullanmak
- `break`, `continue`, `pass` ve iç içe döngülerle çalışmak

Konu anlatımı

Hatırlatma: if-else ile fazla mesai (Slayt 1–3)

40 saati aşan saatlerin 1,5 katı ücretle ödendiği bir maaş hesabı, iki yollu bir karardır:

PYTHON
```
hours = 45
pay_rate = 20
if hours > 40:
    gross = 40 * pay_rate + (hours - 40) * pay_rate * 1.5
else:
    gross = hours * pay_rate
print(gross)
```

ÇIKTI
```
950.0
```

İç içe karar yapıları (Slayt 4–9)

Bir karar yapısı başka bir karar yapısının içine yerleştirilebilir. Örnek: kredi için **hem** yılda en az 30.000$ kazanmak **hem de** en az 2 yıldır çalışıyor olmak gerekir. Önce ilk koşul, o doğruysa ikinci koşul kontrol edilir.

`else` cümlesinin **hangi `if`'e ait olduğunu girinti belirler**: `else`, eşleştiği `if` ile aynı hizada yazılır.

PYTHON

```python
salary = 35000
years_on_job = 1
if salary >= 30000:
    if years_on_job >= 2:
        print('You qualify for the loan.')
    else:
        print('You must have been employed for at least 2 years.')
else:
    print('You must earn at least $30,000 per year.')
```

ÇIKTI

```
You must have been employed for at least 2 years.
```

if-elif-else (Slayt 10–15)

İç içe if-else'in derinleşen girintisini önler. Koşullar **yukarıdan aşağı** denenir; **ilk doğru olan** bloğu çalışır, gerisi atlanır. Hiçbiri doğru değilse `else` çalışır. İstediğin kadar `elif` eklenebilir.

PYTHON

```python
for temp in [120, 36, -5]:
    if temp > 99:
        print(temp, 'water is at boiling temperature')
    elif temp >= 1 and temp <= 99:
        print(temp, 'water temperature is at normal level')
    else:
        print(temp, 'water is at freezing temperature')
```

ÇIKTI

```
120 water is at boiling temperature
36 water temperature is at normal level
-5 water is at freezing temperature
```

Tekrar yapıları ve while (Slayt 16–21)

Aynı kodu kopyalamak programı büyütür ve düzeltmeyi zorlaştırır. **Tekrar yapısı** (döngü) bunu çözer. İki tür vardır: **koşul kontrollü** ve **sayı kontrollü**.

`while koşul:` koşul doğru olduğu sürece gövdeyi tekrarlar. Gövdenin bir kez çalışmasına **iterasyon** denir. `while` **ön testli** bir döngüdür: koşul baştan yanlışsa gövde hiç çalışmaz.

PYTHON

```python
i = 0
while i < 5:
    print(i, end=' ')
```

PYTHON (DEVAM)

```
    i += 1
print()
print('döngüden sonra i =', i)
```

ÇIKTI

```
0 1 2 3 4
döngüden sonra i = 5
```

Döngünün içinde koşulu eninde sonunda yanlış yapan bir şey olmalıdır; yoksa **sonsuz döngü** oluşur (ör. `i += 1` unutulursa).

for döngüsü ve range (Slayt 22–35)

`for degisken in dizi:` dizideki **her öğe için bir kez** döner. Her iterasyonun başında sıradaki öğe **hedef değişkene** atanır. Liste, string, tuple gibi yinelenebilir nesneler üzerinde çalışır.

PYTHON

```
for s in 'Star':
    print(s, end=' ')
print()
for renk in ['kırmızı', 'yeşil']:
    print(renk)
```

ÇIKTI

```
S t a r
kırmızı
yeşil
```

`range` sayı dizileri üretir: `range(stop)` 0'dan başlar; `range(start, stop)`'ta **stop dahil değildir**; üçüncü argüman adımdır, negatif adım geriye sayar.

PYTHON

```
print(list(range(5)), list(range(2, 6)), list(range(0, 10, 3)))
for n in range(1, 4):
    print(n, n ** 2)
print(list(range(19, 11, -1)))
```

ÇIKTI

```
[0, 1, 2, 3, 4] [2, 3, 4, 5] [0, 3, 6, 9]
1 1
2 4
3 9
[19, 18, 17, 16, 15, 14, 13, 12]
```

Birikimli toplam ve artırılmış atama (Slayt 36–38)

Bir serinin toplamı için **döngü** ve 0 ile başlayan bir **biriktirici** (accumulator) gerekir.

PYTHON
```python
total = 0
for n in [5, 10, 15]:
    total += n
print(total)
```

ÇIKTI
```
30
```

Operatör	Örnek	Eşdeğeri
`+=`	`x += 5`	`x = x + 5`
`-=`	`y -= 2`	`y = y - 2`
`*=`	`z *= 10`	`z = z * 10`
`/=`, `//=`, `%=`, `**=`	`a /= b`	`a = a / b`

Sentinel ve girdi doğrulama (Slayt 39–41)

Sentinel, bir öğe dizisinin sonunu işaretleyen özel değerdir ve normal bir değerle karışmayacak kadar ayırt edici olmalıdır. **GIGO** (garbage in, garbage out): kötü girdi kötü çıktı üretir. **Girdi doğrulama**, girdiyi işlemeden önce incelemektir; genellikle girdi kötü olduğu sürece dönen bir `while` döngüsüyle yapılır.

PYTHON
```python
girdiler = [12, 7, -1, 99]   # -1 sentinel: listenin sonunu gösterir
toplam = 0
for x in girdiler:
    if x == -1:
        break
    toplam += x
print(toplam)

denemeler = ['-5', 'abc', '30']   # klavyeden sırayla gelen girdiler gibi
sira = 0
yas = int(denemeler[sira]) if denemeler[sira].isdigit() else -1
while yas < 0:
    print('Geçersiz girdi:', denemeler[sira])
    sira += 1
    yas = int(denemeler[sira]) if denemeler[sira].isdigit() else -1
print('Kabul edilen yaş:', yas)
```

ÇIKTI

```
19
Geçersiz girdi: -5
Geçersiz girdi: abc
Kabul edilen yaş: 30
```

break, continue ve pass (Slayt 42–50)

- `break` içinde bulunduğu döngüyü **tamamen bitirir**; program döngüden sonraki satırdan devam eder.
- `continue` o iterasyonun **geri kalanını atlar**, sıradaki iterasyona geçer.
- `pass` hiçbir şey yapmaz; söz dizimi bir ifade gerektirip kod istemediğinde kullanılır.

PYTHON

```python
for w in 'gamer':
    if w == 'm':
        break
    print(w, end=' ')
print('| The loop has ended.')
for i in range(10):
    if i % 2 != 0:
        continue
    print(i, end=' ')
print()
for i in range(3):
    pass
print('pass bir şey yapmadı, i =', i)
```

ÇIKTI

```
g a | The loop has ended.
0 2 4 6 8
pass bir şey yapmadı, i = 2
```

İç içe döngüler (Slayt 51–56)

İç döngü, dış döngünün **her** iterasyonunda tüm iterasyonlarını tamamlar (analog saatte yelkovan ve saniye kolu gibi). Toplam iterasyon = iç × dış.

PYTHON

```python
for i in range(2):
    for j in range(3):
        print(i, ':', j, end='   ')
print()
sayac = 0
for saat in range(24):
    for dakika in range(60):
```

PYTHON (DEVAM)

```
        sayac += 1
print(sayac)
```

ÇIKTI

```
0 : 0   0 : 1   0 : 2   1 : 0   1 : 1   1 : 2
1440
```

Sık yapılan hatalar

- `else`**'i yanlış** `if` **ile hizalamak:** iç içe yapılarda mantığı tamamen değiştirir.
- `elif` **yerine** `else koşul:` **yazmak** → `SyntaxError`; `else` koşul almaz.

PYTHON

```
t = 50
if t > 99:
    print('boiling')
else t > 0:
    print('normal')
```

ÇIKTI

```
SyntaxError: invalid syntax
```

- `while`**'da koşulu değiştirmeyi unutmak** → sonsuz döngü.
- `range`**'in bitişi dahil sanmak:** `range(1, 5)` 1, 2, 3, 4 üretir.
- **Tam sayının üzerinde dönmeye çalışmak:** `for i in 5:` → `TypeError`; `range(5)` gerekir.
- **Biriktiriciyi döngünün içinde sıfırlamak:** `total = 0` döngüden önce yazılmalı.

Kısa tekrar soruları

1. `if-elif-else` yapısında birden çok koşul doğruysa hangi blok çalışır?
2. `while` neden "ön testli" bir döngüdür?
3. `list(range(10, 0, -3))` nedir?
4. `break` ile `continue` arasındaki fark nedir?
5. Dış döngü 4, iç döngü 5 kez dönüyorsa iç gövde toplam kaç kez çalışır?
6. Sentinel değeri seçerken neye dikkat edilir?

Cevaplar

1. Yalnızca ilk doğru koşulun bloğu.
2. Koşulu her iterasyondan önce test eder; koşul baştan yanlışsa gövde hiç çalışmaz.
3. `[10, 7, 4, 1]`
4. `break` döngüyü tamamen bitirir; `continue` yalnızca o iterasyonun kalanını atlayıp sıradakine geçer.
5. 4 × 5 = 20 kez.
6. Normal veri değerleriyle karışmayacak kadar ayırt edici olmalıdır.